

Sistema de Detecção de Intrusão Baseado em xAI para Detecção de Ataques DNS over HTTPS (DoH)

1st Daniel Nascimento da Silva
Centro de Informática - UFPE
Recife, Brazil
dns3@cin.ufpe.br

2nd Lucas Ferreira Alves
Centro de Informática - UFPE
Recife, Brazil
lfa4@cin.ufpe.br

3rd Maria Carolina Santos Berrafato
Centro de Informática - UFPE
Recife, Brazil
mcsb3@cin.ufpe.br

4th Millena Ferreira Marçal das Neves
Centro de Informática - UFPE
Recife, Brazil
mfmn@cin.ufpe.br

Resumo — Com a adoção do protocolo DNS over HTTPS (DoH), consultas DNS passaram a ser criptografadas dentro do tráfego HTTPS, aumentando a privacidade dos usuários, mas reduzindo a visibilidade dos administradores de rede sobre esse tráfego. Como consequência, a identificação de atividades maliciosas, como malware e tunelamento DNS, torna-se mais desafiadora. Nesse contexto, este trabalho analisa a solução de artigo trabalhado na disciplina IF848 - Detecção de Intrusão, baseada em Inteligência Artificial Explicável (xAI), para detecção de tráfego DoH malicioso utilizando o conjunto de dados público CIRA-CIC-DoHBrw-2020. Além de replicar a metodologia no mesmo dataset e em um diferente, o CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD, o presente trabalho propõe melhorias na arquitetura do artigo de referência, visando explorar se há consistência nas métricas de performance e explicabilidade dos modelos de ML na tarefa de detecção de tráfego DoH malicioso, e as respectivas ferramentas de tunelamento que o geraram. Obtivemos precisão maior que 92% para ataques gerados por todas as ferramentas presentes no dataset, exceto dnscat2.

Palavras-chave — DNS over HTTPS, Detecção de Intrusão, Inteligência Artificial Explicável, Random Forest, SHAP, Segurança Cibernética.

Abstract—With the adoption of the DNS over HTTPS (DoH) protocol, DNS queries began to be encrypted within HTTPS traffic, increasing user privacy while reducing network administrators' visibility into such traffic. As a consequence, the identification of malicious activities, such as malware and DNS tunneling, becomes more challenging. In this context, this work analyzes the solution proposed in a paper studied in the IF848 - Intrusion Detection course, based on Explainable Artificial Intelligence (xAI), for the detection of malicious DoH traffic using the public CIRA-CIC-DoHBrw-2020 dataset. In addition to replicating the methodology on the same dataset and on a different one, namely CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD, the present work proposes improvements to the architecture of the reference paper, aiming to investigate whether there is consistency in the performance and explainability metrics of ML models in the task of detecting malicious DoH traffic and the respective tunneling tools that generated it. We achieved precision greater than 92% for attacks generated by all tunneling tools present in the dataset, except for dnscat2.

Keywords — DNS over HTTPS, Intrusion Detection, Explainable AI, Random Forest, SHAP, Cybersecurity.

I. INTRODUÇÃO

O Sistema de Nomes de Domínio (DNS) permanece como um dos vetores de ataque mais críticos e prevalentes na segurança cibernética atual, caracterizando-se por sua posição estratégica como ponto de entrada para invasão de redes e exfiltração de dados. Segundo o relatório global de ameaças DNS da EfficientIP e IDC (2021)[1], cerca de 87% das organizações pesquisadas sofreram ataques DNS naquele ano, um aumento de 8 pontos percentuais em relação a 2020. Esse número evidencia a relevância do tema, uma vez que o DNS continua sendo uma superfície de ataque amplamente explorada e com potencial para gerar impactos financeiros e operacionais significativos nas organizações.

As abordagens tradicionais de defesa dependem fortemente da inspeção de tráfego DNS em texto claro, permitindo que firewalls e sistemas de detecção de intrusão (IDS) analisem os domínios consultados para identificar comportamentos suspeitos, como técnicas de tunelamento DNS utilizadas por malwares para estabelecer canais encobertos de comando e controle (CC). Esse modelo de defesa, entretanto, baseia-se na premissa de que o conteúdo das consultas permanece visível ao defensor. Com a introdução do protocolo DNS over HTTPS (DoH), desenvolvido para aumentar a privacidade dos usuários por meio da criptografia das consultas DNS dentro de conexões HTTPS convencionais, essa visibilidade é reduzida. Como consequência, as mesmas propriedades que fortalecem a privacidade do usuário podem ser exploradas por atacantes para ocultar canais de comunicação maliciosos dentro de tráfego aparentemente legítimo, tornando menos eficazes os mecanismos tradicionais de inspeção baseados em conteúdo.

Diante desse cenário, a comunidade de pesquisa passou a investigar métodos de detecção de tráfego DoH malicioso baseados em características estatísticas dos fluxos de rede que permanecem observáveis mesmo sob criptografia TLS. Diversos trabalhos recentes aplicaram técnicas de aprendizado

de máquina sobre o *dataset* público CIRA-CIC-DoHBrw-2020 com esse objetivo, alcançando resultados promissores na distinção entre tráfego legítimo e malicioso. Contudo, a literatura ainda apresenta desafios relacionados à reprodutibilidade experimental, à comparação padronizada entre modelos e, principalmente, à limitada interpretabilidade das soluções propostas.

Essa falta de interpretabilidade possui implicações práticas importantes. Em ambientes operacionais de segurança cibernética, decisões automatizadas sem justificativa reduzem a confiança dos analistas humanos e dificultam a validação dos alertas produzidos pelos sistemas de detecção. Além disso, a compreensão dos fatores que influenciam as previsões é fundamental para identificar possíveis falhas, vieses ou comportamentos inesperados dos modelos empregados. Dessa forma, embora o DoH imponha restrições à inspeção direta do conteúdo das consultas, isso não elimina a necessidade de transparência sobre os critérios utilizados pelos classificadores para identificar atividades suspeitas. Nesse contexto, observa-se uma lacuna na literatura referente à aplicação de técnicas de Inteligência Artificial Explicável (XAI) especificamente voltadas para a detecção de tráfego DoH malicioso.

Com o objetivo de enfrentar essa limitação, Zebin, Rezvy e Luo [2], em trabalho publicado na IEEE Transactions on Information Forensics and Security, propõem uma solução que combina alto desempenho de classificação com mecanismos de explicabilidade. A abordagem utiliza um classificador Random Forest balanceado e empilhado (stacking), treinado sobre o *dataset* CIRA-CIC-DoHBrw-2020, enquanto a interpretação das decisões é realizada por meio da biblioteca SHAP, cujos resultados são apresentados em um dashboard interativo. O presente relatório analisa e contextualiza essa proposta, discutindo sua metodologia, seus resultados e suas contribuições para a área de detecção de intrusão em tráfego DNS criptografado.

As principais contribuições do artigo analisado são:

- 1) **Detecção explicável de tráfego DoH:** Desenvolvimento de uma das primeiras soluções baseadas em Inteligência Artificial Explicável (XAI) voltadas especificamente para a detecção e classificação de tráfego DNS over HTTPS.
- 2) **Classificador Random Forest balanceado e empilhado:** Utilização de três submodelos treinados sobre subconjuntos balanceados via SMOTE e combinados por um meta-classificador de regressão logística, resultando em ganhos de desempenho e redução do tempo de treinamento em comparação com abordagens convencionais.
- 3) **Dashboard de explicação interativo:** integração de mecanismos de explicabilidade por meio de visualizações globais e locais, incluindo gráficos de contribuição das variáveis e análises individuais das previsões.
- 4) **Identificação da origem do tráfego malicioso:** Demonstração de que as mesmas características estatísticas utilizadas na classificação permitem inferir,

com boa precisão, qual ferramenta de tunelamento DNS foi responsável pela geração do tráfego analisado.

II. TRABALHOS RELACIONADOS

A detecção de anomalias em tráfego DNS, especialmente em cenários com criptografia como DNS over HTTPS (DoH), tem evoluído significativamente nos últimos anos. As abordagens transitam desde métodos aplicados ao DNS tradicional até técnicas modernas baseadas em aprendizado de máquina para análise de tráfego criptografado.

A. Detecção em DNS Tradicional e Mitigação de DDoS

Inicialmente, os estudos focaram na análise de tráfego DNS não criptografado e na mitigação de ataques distribuídos de negação de serviço (DDoS). Nesse contexto, o trabalho de Trejo et al. [3] propõe o DNS-ADVP, uma plataforma que combina aprendizado de máquina com visualização interativa para monitoramento de servidores DNS de nível superior (TLD). A abordagem integra técnicas de classificação com modelos visuais que permitem identificar padrões anômalos em tempo real.

Além disso, os autores analisam técnicas clássicas de mitigação, como limitação de requisições por IP e Response Rate Limiting (RRL), demonstrando sua eficácia na redução de tráfego malicioso. No entanto, essas abordagens são limitadas por sua dependência de regras heurísticas e por não considerarem cenários com tráfego criptografado, como o DoH, o que reduz sua aplicabilidade em ambientes modernos.

B. Detecção de DNS over HTTPS com Aprendizado de Máquina

Com a introdução do DoH, a visibilidade do tráfego DNS tornou-se limitada devido à criptografia, exigindo novas abordagens. O trabalho de Vekshin et al. [4] investiga a detecção de tráfego DoH a partir de fluxos HTTPS utilizando aprendizado de máquina. Os autores exploram características estatísticas, como tempo entre pacotes, tamanho dos pacotes e padrões de comunicação, alcançando acurácia superior a 99,9

Além disso, a abordagem permite diferenciar clientes DoH com base em padrões comportamentais. Entretanto, a principal limitação está na forte dependência da engenharia de atributos e da construção de *datasets* específicos, o que pode comprometer a generalização do modelo em cenários reais e dinâmicos.

C. Classificação de Tráfego Malicioso em DoH

Trabalhos mais recentes ampliam o uso de aprendizado de máquina para classificação de tráfego malicioso em ambientes DoH. Jafar et al. [5] realizam uma análise comparativa entre diferentes algoritmos, como Random Forest, Decision Tree e SVM, utilizando o *dataset* CIRA-CIC-DoHBrw-2020. Os resultados demonstram alta acurácia, chegando a valores próximos de 99,99

Apesar do desempenho elevado, essas abordagens apresentam limitações relacionadas à interpretabilidade dos modelos e à dependência de *datasets* específicos. Além disso, modelos altamente otimizados podem sofrer com overfitting, reduzindo

Tabela I
COMPARAÇÃO DOS TRABALHOS RELACIONADOS

Trabalho	Método	Vantagens	Desvantagens
DNS-ADVP	ML + Visualização	Monitoramento em tempo real; análise de tráfego DNS	Não trata DoH; depende de dados
DoH Insight	ML (<i>features</i> de tráfego)	Alta acurácia e distingue clientes DoH	Dependência de <i>features</i> e baixa generalização
Analysis malicious dns queries	RF, DT, SVM	Alta acurácia e comparação de modelos	Baixa interpretabilidade e possível overfitting
Artigo Base	ML + XAI	Alta acurácia, Interpretabilidade e maior transparência	Maior custo computacional e complexidade

sua eficácia em cenários desconhecidos ou ataques mais sofisticados.

D. Inteligência Artificial Explicável para Detecção de DoH

Diante das limitações das abordagens tradicionais, estudos recentes têm explorado o uso de Inteligência Artificial Explicável (XAI) para melhorar a transparência dos modelos de detecção. O artigo base deste trabalho propõe um sistema de detecção de intrusão para tráfego DoH que combina aprendizado de máquina com técnicas de explicabilidade.

A principal contribuição dessa abordagem é permitir não apenas a identificação de ataques, mas também a interpretação das decisões do modelo, aumentando a confiabilidade e facilitando a análise por especialistas. No entanto, essa abordagem pode introduzir maior complexidade computacional e dependência de métodos adicionais para geração de explicações.

Um resumo comparativo das abordagens apresentadas é mostrado na Tabela I.

III. MODELO DE AMEAÇA

O cenário de ameaça abordado neste trabalho concentra-se em ataques de tunelamento DNS (DNS tunneling) realizados sobre o protocolo DNS over HTTPS (DoH), nos quais o canal DNS criptografado é utilizado como meio encoberto de comunicação entre um host comprometido e um servidor controlado pelo atacante. Nesse contexto, o protocolo DoH fornece uma camada adicional de ocultação ao encapsular consultas DNS dentro de conexões HTTPS protegidas por TLS, reduzindo a visibilidade dos mecanismos tradicionais de monitoramento de rede.

A. Premissas do ataque

Para a execução do ataque, assumem-se as seguintes premissas em relação às capacidades do adversário:

- 1) **Controle de infraestrutura própria:** o atacante registra um domínio sob seu controle e opera um servidor de tunelamento que, embora se apresente como servidor DNS autoritativo, funciona efetivamente como um servidor de comando e controle (CC).

- 2) **Comprometimento prévio do host:** a máquina-alvo encontra-se previamente infectada com uma ferramenta de tunelamento DNS, como dns2tcp, DNSCat2 ou Iodine.
- 3) **Permissividade da rede:** a rede da vítima permite conexões DoH para resolvedores públicos, como Cloudflare, Google, Quad9 ou AdGuard, cenário cada vez mais comum devido à adoção crescente do protocolo por navegadores modernos.
- 4) **Opacidade do canal criptografado:** os mecanismos de defesa da organização (firewalls, IDS ou IPS) não possuem acesso ao conteúdo das consultas DNS encapsuladas em TLS e, portanto, não conseguem aplicar inspeção baseada em payload.

B. Análise comportamental do Ataque e Formalização

Embora o artigo original não apresente uma formalização matemática explícita do modelo de ameaça, é possível representar conceitualmente o cenário descrito pelos autores para evidenciar quais informações permanecem acessíveis ao defensor após a adoção do DoH.

Seja um fluxo de tráfego DoH representado por $F = (P, S)$, onde P representa o conteúdo da comunicação DNS encapsulado e protegido por TLS, enquanto $S = (s_1, \dots, s_{29})$ corresponde ao conjunto de características estatísticas observáveis do fluxo, incluindo métricas de duração, volume de bytes, comprimento de pacotes e tempos entre transmissões. Como o conteúdo DNS encontra-se criptografado, a observação realizada pelo defensor fica restrita às características estatísticas do fluxo:

$$Obs_{defensor}(F) = S, \text{ com } Obs_{defensor}(P) = \phi \quad (1)$$

Em outras palavras, diferentemente do DNS tradicional em texto claro, o conteúdo das consultas e respostas não está disponível para inspeção direta. Essa limitação constitui a principal diferença estrutural do problema investigado neste trabalho.

Durante o ataque, informações de interesse do adversário são fragmentadas e incorporadas a registros DNS utilizados pela ferramenta de tunelamento. Essas informações são posteriormente encapsuladas dentro do canal HTTPS e transmitidas ao resolvedor DoH, tornando-se indistinguíveis do restante do tráfego criptografado sob uma análise baseada exclusivamente em conteúdo.

Seja ainda $C: S \rightarrow \text{NonDoH, Benign-DoH, Malicious-DoH}$ o classificador utilizado pelo sistema de detecção, responsável por inferir a classe de um fluxo a partir apenas das características estatísticas observáveis. Define-se o sucesso do ataque, do ponto de vista do mecanismo de detecção, como a condição em que o fluxo malicioso não recebe a classificação correspondente à sua verdadeira natureza:

$$\text{Sucesso(ataque)} \iff P(\text{Malicious-DoH} \mid S_{\text{mal}}) < \tau_{\text{alerta}} \quad (2)$$

ou seja, o ataque é bem-sucedido quando a probabilidade atribuída pelo classificador à classe maliciosa, dado o vetor

estatístico S_{mal} do fluxo de ataque, permanece abaixo do limiar operacional definido pelo sistema para geração de alertas, fazendo com que o fluxo seja tratado como tráfego benigno ou Non-DoH.

A efetividade do ataque depende, entretanto, de um equilíbrio entre furtividade e capacidade de comunicação. Para transmitir uma quantidade útil de dados, as ferramentas de tunelamento precisam gerar um volume mínimo de consultas DNS, o que inevitavelmente altera características observáveis do fluxo de rede. Como consequência, padrões estatísticos relacionados à duração das conexões, distribuição dos tamanhos de pacotes, quantidade de bytes transmitidos e intervalos temporais entre pacotes tendem a diferir daqueles observados em tráfego DoH legítimo. Essas diferenças não revelam o conteúdo transportado pelo túnel, mas podem fornecer evidências indiretas da existência de um canal de comunicação encoberto. A hipótese central explorada pelos autores é que tais padrões estatísticos são suficientes para distinguir tráfego DoH malicioso de tráfego legítimo mesmo sem acesso ao payload criptografado. O sistema apresentado no artigo busca justamente aprender essas diferenças a partir das características observáveis do fluxo, transformando variações estatísticas sutis em sinais úteis para detecção.

C. Ferramentas de tunelamentos

Para avaliar a capacidade de detecção de tráfego malicioso encapsulado, foi utilizado ataques DoH utilizando outras técnicas de tunelamento dns. Esses ataques permitem a criação de canais encobertos para retirada de dados, comunicação com servidores e evasão de mecanismos tradicionais de segurança.

Dns2tcp: É uma ferramenta cujo o seu objetivo é permitir a utilização de serviços TCP mesmo em ambientes onde apenas o tráfego DNS é permitido. A ferramenta implementa um modelo cliente-servidor e utiliza registros DNS para transportar os dados encapsulados. [6]

Dnscat2: Foi projetado para estabelecer canais bidirecionais de comunicação entre um host comprometido e um servidor remoto, utilizando consultas DNS. É frequentemente associado a cenários de pós-exploração, permitindo controle remoto e transferência de dados.[7]

Iodine: Implementa um túnel IP sobre DNS permitindo o transporte de pacotes IP completos através de consultas DNS. Seu diferencial é o suporte a diferentes tipos de registros DNS e mecanismos de codificação que aumentam a taxa de transferência do canal encoberto.[7]

DNSTT: Representa uma geração mais recente de ferramentas de tunelamento. Diferentemente das soluções tradicionais, o DNSTT pode operar sobre o DNS over HTTPS e DNS over TLS, aproveitando a criptografia para dificultar ainda mais a fiscalização do tráfego.[8]

TCP-over-DNS: Tem como objetivo transportar fluxos TCP inteiros através do protocolo DNS. Essa abordagem permite a utilização de aplicações convencionas sobre canais DNS, ampliando a capacidade de evasão em ambiente com políticas restritivas de firewall.[6]

TUNS: É uma implementação de IP-over-DNS focada em simplicidade e conformidade com o protocolo DNS. Ela prioriza a compatibilidade com diferentes infraestruturas de rede e condições degradadas de comunicação.[6]

Essas ferramentas representam diferentes estratégias de encapsulamento de dados em DNS, variando desde o transporte de conexões TCP até o tunelamento completo de tráfego IP. A diversidade de técnicas empregadas por essas soluções justifica sua consideração no modelo de ameaça adotado neste trabalho, uma vez que elas produzem padrões de tráfego distintos e representam desafios relevantes para sistemas de detecção baseados em aprendizado de máquina.

IV. SISTEMA PROPOSTO PELO ARTIGO DE REFERÊNCIA

O artigo propõe um pipeline de detecção em quatro estágios funcionais: preparação do *dataset*, pré-processamento, treinamento/validação cruzada, e geração de explicações via XAI, culminando em um classificador denominado Balanced Stacked Random Forest.

A. Arquitetura Geral

De forma resumida, a entrada do sistema consiste em fluxos de rede representados por 29 características estatísticas extraídas do *dataset* CIRA-CIC-DoHBrw-2020 (estatísticas de duração, bytes de fluxo, comprimento de pacote, tempo de pacote e atraso entre pacotes). Essas características passam por etapas de normalização e balanceamento antes de alimentarem três classificadores Random Forest independentes, treinados em paralelo sobre subconjuntos distintos dos dados.

As previsões desses três sub-modelos são então combinadas por um meta-classificador de Regressão Logística, responsável pela decisão final (etapa de stacking). Como saída, o sistema produz a classe prevista para o fluxo (Non-DoH, Benign-DoH ou Malicious-DoH), a probabilidade associada a essa predição e um conjunto de explicações geradas pelo SHAP que justificam a decisão tomada, exibidas em gráficos de contribuição e tabelas no dashboard interativo.

A arquitetura é composta pelos seguintes componentes principais:

- 1) **dataset mesclado de três classes:** combinação de uma camada Non-DoH vs. DoH com outra Benign-DoH vs. Malicious-DoH, totalizando 889.809 instâncias Non-DoH, 19.746 Benign-DoH e 249.553 Malicious-DoH (proporção original aproximada de 45:1:12).
- 2) **Camada de pré-processamento/limpeza:** análise exploratória, seleção de *features*, codificação logarítmica de valores grandes, normalização Min-Max e divisão treino/teste.
- 3) **Camada de treinamento, validação cruzada e avaliação:** três sub-modelos *Random Forest* treinados em paralelo sobre subconjuntos balanceados, combinados por um meta-classificador via *stacking*.
- 4) **Camada de XAI:** aplicação do *SHAP TreeExplainer* sobre o modelo final, com saída em matriz de confusão, gráficos de contribuição e *dashboard* interativo.

B. Pré-processamento e Balanceamento

Todas as *features* numéricas são normalizadas por escala Min-Max:

$$x_{norm} = \frac{(x - \min)}{(\max - \min)} \quad (3)$$

A equação é ajustada apenas no conjunto de treino (90% dos dados) e aplicada ao conjunto de teste (10%) com os mesmos parâmetros.

Para lidar com o forte desbalanceamento de classes, os 90% de dados de treino são divididos em três subconjuntos. Cada subconjunto usa uma fração distinta do tráfego Non-DoH, compartilha as mesmas amostras maliciosas entre as três divisões e aplica *SMOTE* para superamostrar a classe benigna, minoritária. O resultado é uma proporção aproximada de 15:12:12 por subconjunto, bem mais equilibrada que a original, exigindo menos dados sintéticos por sub-modelo do que um balanceamento único sobre todo o conjunto.

C. Classificadores Base: Random Forest

Cada sub-modelo é um Random Forest com *numTrees* = 10 árvores e 28 *features* candidatas por nó, limitados a uma profundidade máxima de 5. Em cada nó *t*, a qualidade de uma divisão candidata é avaliada pelo índice de impureza de Gini:

$$Gini(t) = 1 \sum_{i=1}^k p \quad (4)$$

Onde *p* é a proporção de amostras da classe *i* presentes no nó *t*, considerando as *k* = 3 classes do problema (Non-DoH, Benign-DoH, Malicious-DoH). A divisão escolhida em cada nó é aquela que produz a maior redução do índice de Gini entre o nó pai e os nós filhos resultantes, ou seja, a divisão que mais aumenta a pureza (homogeneidade de classe) dos subconjuntos gerados. A classificação final de uma amostra resulta de votação majoritária entre as árvores do sub-modelo. Esse mesmo critério de redução de Gini é reaproveitado pelos autores na etapa de seleção de *features* do pré-processamento, e é também a métrica de impureza sobre a qual o *TreeExplainer* do SHAP se apoia para calcular a contribuição marginal de cada variável nas predições do modelo. A Equação 4 é, portanto, o elo entre a capacidade discriminativa do classificador e a capacidade explicativa do sistema. A implementação usa o *RandomForest Classifier* do *scikit-learn*.

D. Camada de Empilhamento (Stacking)

As saídas dos três sub-modelos Random Forest são usadas como novas *features* de entrada para um meta-classificador de Regressão Logística, via *StackingClassifier*, produzindo a decisão final do sistema. O processo completo é resumido no Algoritmo 1, reportado pelos autores:

Algoritmo 1: Treinamento do Balanced Stacked Random Forest.

Entrada: $X = x_1, \dots, x_m$, 29 *features*, rótulos de treino

Pré-processamento:

- 1) Normalizar via $x_{norm} = \frac{(x - \min)}{(\max - \min)}$
- 2) Criar subconjuntos balanceados e aplicar SMOTE

Classificador base:

- 3) Inicializar Random Forest (10 árvores)
- 4) Treinar sub-modelos em paralelo
- 5) Calcular rótulos para cada amostra (critério de Gini)
- 6) Validação cruzada de 10 folds (otimização de hiper-parâmetros)

Camada de Stacking:

- 7) Empilhar sub-modelos via meta-classificador (Regressão Logística)

Geração de saída XAI:

- 8) Aplicar TreeExplainer (SHAP) para gerar gráfico e tabela de contribuição.

Saída: rótulos de classe, probabilidade, gráfico e tabela de contribuição

E. Camada de Explicabilidade (XAI)

Sobre o modelo final é aplicado o TreeExplainer do SHAP, eficiente para modelos baseados em árvore, gerando três tipos de saída:

- 1) gráficos de resumo global, ordenando as 29 *features* pelo impacto médio absoluto na decisão, destacando duração do fluxo e *features* de comprimento/variância de pacote como as mais relevantes, alinhadas às diferenças estatísticas observadas entre tráfego legítimo e malicioso;
- 2) gráficos de dependência, relacionando o valor de uma *feature* ao seu valor SHAP correspondente, por exemplo, identificando que durações acima de aproximadamente 40 segundos deslocam a decisão do modelo para "malicioso";
- 3) explicações locais por instância, em um dashboard interativo exibindo probabilidade por classe, tabela de contribuição por *feature* e gráfico em cascata (*waterfall plot*).

F. Justificativa da Abordagem

A escolha de Random Forest se justifica por sua alta acurácia como método supervisionado e por ser nativamente compatível com explicadores baseados em árvore (*TreeExplainer*), que são exatos e computacionalmente eficientes. O esquema de sub-modelos balanceados com stacking reduz o volume de dados sintéticos gerados por SMOTE em cada sub-modelo e permite paralelismo no treinamento, justificando a redução de três vezes no tempo total relatada pelos autores. A Regressão Logística foi escolhida como meta-classificador por sua simplicidade e interpretabilidade na camada final de decisão. Por fim, o SHAP foi escolhido por se basear nos valores de Shapley da teoria dos jogos, oferecendo atribuição matematicamente fundamentada da contribuição de cada *feature*, exatamente a transparência ausente em soluções anteriores e essencial em cenários criptografados.

V. SOLUÇÃO PROPOSTA PELA EQUIPE PARA MELHORAR A SOLUÇÃO DO ARTIGO DE REFERÊNCIA

Embora o objetivo central desta frente do projeto tenha sido a reprodução fiel do classificador Balanced Stacked Random Forest proposto por [2], o volume do conjunto de dados (aproximadamente 1,16 milhão de instâncias após limpeza) e o desbalanceamento extremo entre classes (proporção aproximada de 45:1:12 entre Non-DoH, Benign-DoH e Malicious-DoH) exigiram decisões de engenharia não detalhadas explicitamente pelos autores originais.

Em particular, o Algoritmo 1 do artigo original limita-se a indicar que "a validação cruzada de dez dobras é realizada de forma supervisionada para otimização do modelo e ajuste de parâmetros", sem especificar como as previsões dessa validação cruzada alimentam o meta-classificador da camada de stacking. Para eliminar essa ambiguidade e evitar vazamento de dados (*data leakage*) entre a camada base e o meta-classificador, a equipe implementou explicitamente um esquema de previsões *Out-of-Fold* (OOF), descrito no Algoritmo 2.

Algoritmo 2: Geração de Previsões *Out-of-Fold* (OOF) para o Meta-Classificador

Entrada: X_{train} (90% dos dados), y_{train} , $k = 10$ dobras

- 1) Inicializar matriz X_{meta} ($m \times 9$), $m = |X_{train}|$
- 2) Particionar X_{train} em 10 dobras estratificadas (StratifiedKFold)
- 3) Para cada dobra $i = 1, \dots, 10$: $label = (c)$
 - a) Separar dobra i como validação; demais 9 dobras como treino interno
 - b) No treino interno, dividir Non-DoH em 3 blocos paralelos
 - c) Aplicar SMOTE em Benign-DoH (igualar a Malicious-DoH) em cada bloco
 - d) Treinar 3 sub-modelos Random Forest (10 árvores, profundidade 5), um por bloco
 - e) Prever probabilidades dos 3 sub-modelos sobre a dobra i (nunca vista pelos sub-modelos)
 - f) Armazenar as 9 probabilidades (3 modelos $\times$ 3 classes) nas posições da dobra i em X_{meta}
- 4) Treinar a Regressão Logística (meta-classificador) sobre X_{meta} vs. y_{train} ;
- 5) Re-treinar os 3 sub-modelos oficiais com 100% de X_{train} (mesma lógica de blocos e SMOTE);

Saída: meta-classificador treinado, 3 sub-modelos oficiais

A partir dessa base arquitetural, a equipe testou duas variantes do meta-classificador sobre os mesmos dados do artigo de referência:

- 1) A configuração purista, fiel à descrição original (Regressão Logística padrão);
- 2) Uma variante com o parâmetro $class_weight = 'balanced'$, que pondera a função de perda da Regressão Logística inversamente à frequência de cada classe:

$$w_c = \frac{n}{k \cdot n_c} \quad (5)$$

onde n é o número total de amostras, k é o número de classes e n_c é o número de amostras da classe c no conjunto de treino. Essa ponderação penaliza mais fortemente os erros cometidos na classe minoritária (Benign-DoH) durante o ajuste do meta-classificador, sem alterar o treinamento dos sub-modelos Random Forest da camada base.

Dentre as oportunidades de melhorias e experimentação que identificamos na etapa de seminário da disciplina, a que decidimos propor é voltada para a expansão do escopo de ataques maliciosos: sabe-se que o artigo de referência se baseia no *dataset* mais utilizado para aplicações de detecção de ataques do tipo DoH [9], mas que ele só contém instâncias maliciosas geradas por 3 diferentes ferramentas de tunneling. Por esse motivo, decidimos testar o comportamento da arquitetura proposta para um *dataset* que envolva ataques DoH gerados por 3 novos tipos de ferramentas: *dnstt*, *tcp_over_dns*, e *tuns*. Há uma descrição mais detalhada sobre o conjunto de dados na seção de metodologia deste relatório.

A expansão do escopo de ataques é um experimento que consideramos particularmente interessante por nos permitir testar a consistência na explicabilidade das decisões tomadas pelos algoritmos originais, comparando não apenas métricas quantitativas como precisão, mas fazendo uma análise mais aprofundada sobre quais atributos são mais relevantes na classificação dos ataques. Dessa forma, esperamos que a metodologia utilizada no artigo de referência seja consistente para a detecção de novos tipos de fluxos de rede maliciosos DoH com eficácia, de forma a explorar se pode ser replicada com facilidade ou adaptada a novos contextos, e se a tomada de decisão dos algoritmos também é consistente.

O foco do artigo de referência é classificar fluxos de rede entre Non-DoH, Benign-DoH, e Malicious-DoH. Embora mencionem brevemente testar a classificação das 3 ferramentas de tunneling propriamente ditas, não é um experimento detalhado. Na nossa proposta de melhoria, fizemos uma classificação com 8 classes em vez de 3 ou 5: Non-DoH, Benign-DoH, e todas as 6 ferramentas de tunneling disponíveis no novo *dataset*. Além disso, uma parte particularmente importante da arquitetura original é o balanceamento dos dados, etapa que precisamos adaptar para que se ajustasse ao novo conjunto: originalmente, os autores dividem a classe majoritária em 3 partições de dados, replicando as instâncias de outras classes e utilizando SMOTE para oversampling; por causa da nova proporção entre classes, nós adaptamos a arquitetura para o balanceamento, dividindo a classe majoritária em 5 subconjuntos em vez de 3, e replicando e utilizando SMOTE nas minoritárias de forma similar à solução original. Com essa alteração no particionamento, precisamos também alterar de 3 para 5 o número de sub-classificadores treinados, já que cada subconjunto é alimentado a um modelo diferente para predição das probabilidades e geração dos meta-atributos.

O restante das etapas permanece inalterado. Com nossas alterações, ajustamos o Algoritmo 2 para que descreva nossa proposta com clareza e resulte no Algoritmo 3.

Algoritmo 3: Geração de Predições Out-of-Fold (OOF) para o Meta-Classificador - Nossa Proposta

Entrada: X_{train} (90% dos dados), y_{train} , $k = 10$ dobras

- 1) Inicializar matriz X_{meta} ($m \times 40$), $m = |X_{train}|$
 - 2) Particionar X_{train} em 10 dobras estratificadas (StratifiedKFold)
 - 3) Para cada dobra $i = 1, \dots, 10$: label=(c)
 - a) Separar dobra i como validação; demais 9 dobras como treino interno
 - b) No treino interno, dividir Non-DoH em 5 blocos paralelos
 - c) Aplicar SMOTE em Benign-DoH (igualar a Malicious-DoH) em cada bloco
 - d) Treinar 5 sub-modelos Random Forest (10 árvores), um por bloco
 - e) Prever probabilidades dos 5 sub-modelos sobre a dobra i (nunca vista pelos sub-modelos)
 - f) Armazenar as 40 probabilidades (5 modelos $\times$ 8 classes) nas posições da dobra i em X_{meta}
 - 4) Treinar a Regressão Logística (meta-classificador) sobre X_{meta} vs. y_{train} ;
 - 5) Re-treinar os 5 sub-modelos oficiais com 100% de X_{train} (mesma lógica de blocos e SMOTE);
- Saída:** meta-classificador treinado, 5 sub-modelos oficiais

VI. METODOLOGIA

A. Dados usados pelo artigo de referência

A reprodução empregou o mesmo conjunto de dados utilizado pelos autores do artigo de referência: o CIRA-CIC-DoHBrw-2020. A utilização do mesmo *dataset* teve como objetivo possibilitar uma comparação direta entre os resultados obtidos neste trabalho e aqueles reportados pelos autores.

Os dados foram obtidos a partir dos arquivos l1-nondoh.csv, l2-benign.csv e l2-malicious.csv. O arquivo l1-doh.csv não foi utilizado, uma vez que suas instâncias já se encontram representadas nos conjuntos Benign-DoH e Malicious-DoH. Após a importação, cada registro recebeu o rótulo correspondente à sua classe de origem (Non-DoH, Benign-DoH ou Malicious-DoH), sendo posteriormente consolidado em um único conjunto de dados.

Em seguida, foram removidos atributos de identificação de rede, incluindo endereços IP de origem e destino, portas de origem e destino e timestamps. Essa etapa foi realizada para garantir que o processo de classificação se baseasse exclusivamente nas características estatísticas dos fluxos de rede, conforme proposto pelos autores.

Registros contendo valores nulos ou infinitos foram descartados. Posteriormente, normalização Min-Max, descrita na Equação 3 da Seção IV-B, foi aplicada às 29 características restantes.

A divisão entre treinamento e teste seguiu a proporção de 90% e 10%, respectivamente, conforme descrito no artigo original. Para preservar a distribuição das classes em ambos

os conjuntos, foi empregada amostragem estratificada. Não foi definido um conjunto de validação independente, uma vez que a seleção de hiperparâmetros e a geração das predições utilizadas pelo meta-classificador foram realizadas por meio de validação cruzada de 10 dobras.

A Tabela II apresenta a distribuição final das instâncias entre os conjuntos de treinamento e teste utilizados na reprodução dos experimentos.

Tabela II
DISTRIBUIÇÃO DAS CLASSES NO CONJUNTO DE DADOS

Classe	Treino (90%)	Teste (10%)	Total (réplica)	Total (original)
Non-DoH	800.829	88.981	≈ 889.810	889.809
Malicious-DoH	224.595	24.955	≈ 249.550	249.553
Benign-DoH	17.775	1.975	≈ 19.750	19.746
Total	1.043.199	115.911	$\approx 1.159.110$	1.159.108

A diferença de poucas dezenas de instâncias entre os totais da réplica e os da Tabela I do artigo original deve-se exclusivamente à remoção de linhas com valores nulos ou infinitos durante a etapa de limpeza, confirmando que o conjunto de dados utilizado na reprodução é, na prática, idêntico ao do estudo de referência.

B. Métricas de Avaliação

As métricas utilizadas para comparar a réplica com os resultados originais seguem as mesmas definições empregadas pelos autores: acurácia, precisão, recall e F1-score, definidas a partir de verdadeiros positivos (TP), verdadeiros negativos (TN), falsos positivos (FP) e falsos negativos (FN) por classe.

$$\text{Acurácia} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$\text{Precisão} = \frac{TP}{TP + FP} \quad (7)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

$$F1 = \frac{2 \cdot \text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \quad (9)$$

C. Novo conjunto de dados escolhido

Para replicar os experimentos originais em um novo conjunto de dados, decidimos utilizar a junção do CIRA-CIC-DoHBrw-2020 [10] com o DoH-Tunnel-Traffic-HKD [11]: o CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD [12]. O conjunto de dados combinado foi escolhido em vez do DoH-Tunnel-Traffic-HKD porque entendemos que a arquitetura proposta originalmente no artigo de referência faz mais sentido para *datasets* desbalanceados, o que não é o caso do HKD. Além disso, já que o CIRA-CIC-DoHBrw-2020 possui uma quantidade significativamente maior de fluxos de rede do tipo Non-DoH em comparação com outras classes, sua versão unificada com o DoH-Tunnel-Traffic-HKD nos permitiu testar a técnica proposta em um contexto de desbalanceamento de dados, mas dessa vez com instâncias maliciosas adicionais

geradas por 3 novos tipos de ferramentas de tunneling: dnstt, tcp-over-dns, e tuns. Isso nos permitiu testar a consistência tanto nas métricas de performance em comparação com o trabalho original, quanto na explicabilidade das decisões, além de aumentar o alcance da metodologia a outras ferramentas.

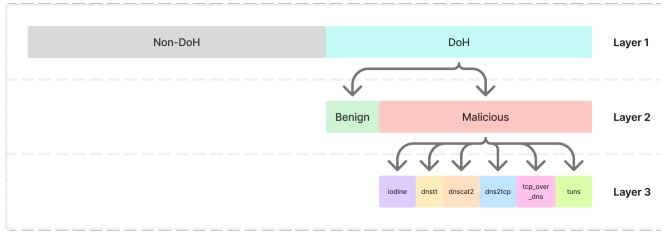


Fig. 1. Divisão de classes por camada do *dataset* CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD

O *dataset* unificado conta com três camadas com diferentes atributos preditores, uma a mais em relação ao CIRA-CIC-DoHBrw-2020, conforme ilustra a Figura 1. A primeira camada é voltada para classificação binária de tráfego DoH (374.803 instâncias) ou Non-DoH (897.493 instâncias); a segunda utiliza apenas as instâncias DoH da primeira camada, detalhando sua classificação entre Benign-DoH (19.807 instâncias) e Malicious-DoH (354.996 instâncias); já a terceira e última camada utiliza apenas as instâncias classificadas como Malicious-DoH da segunda camada, de forma que as possíveis classes sejam as ferramentas de tunneling que geraram o respectivo ataque malicioso: dns2tcp (167.486 instâncias), dnscat2 (35.770 instâncias), iodine (46.580 instâncias), dnstt (46.080 instâncias), tcp-over-dns (30.040 instâncias), e tuns (29.040 instâncias).

Por conter novas instâncias maliciosas, a proporção entre classes para a replicação da metodologia original (classificação entre Non-DoH, Benign-DoH e Malicious-DoH) é 45:1:18, enquanto para a nossa proposta de solução (com 8 classes), a proporção é 45:8:2:2:2:2:1:1. O CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD conta com as mesmas 35 colunas do *dataset* original.

Para replicar os experimentos originais no novo conjunto de dados, unimos as instâncias Non-DoH da primeira camada com a segunda camada (que possui as classes Benign-DoH e Malicious-DoH), de forma que pudéssemos classificar os fluxos de rede entre essas 3 classes, assim como no artigo de referência. Já para a solução proposta, unificamos as instâncias Non-DoH da primeira camada com as instâncias Benign-DoH da segunda camada e com a terceira camada completa, composta pelas 6 classes de instâncias maliciosas, o que nos gerou um problema de classificação com 8 classes. Embora a quantidade de classes seja diferente, a quantidade de instâncias é a mesma nos dois cenários. Além disso, as 5 primeiras colunas do *dataset*, que correspondem a metadados associados aos fluxos de rede, foram removidas nesta etapa. São elas: SourceIP, DestinationIP, SourcePort, DestinationPort, e TimeStamp.

O novo *dataset* muda a proporção de balanceamento original de 45:1:12 para 45:1:18 por acrescentar apenas instâncias maliciosas aos dados. Para replicar o experimento original, conforme já foi previamente mencionado, repetimos as mesmas estratégias para o balanceamento: começamos dividindo os dados em conjuntos de treino (90%) e teste (10%), sendo a primeira etapa para evitar vazamento de dados; no conjunto de treino, foi realizada validação cruzada com 10 dobras. Para cada dobra, as instâncias majoritárias (Non-DoH) dos dados de treino foram divididas em partições como estratégia de balanceamento, e para cada partição, as classes minoritárias foram repetidas e aumentadas (oversampling) usando o SMOTE. Em seguida, normalizamos usando o MinMax Scaler.

1) Pré-processamento na Replicação da Metodologia

Original: 3 Classes Os autores dividem a classe majoritária (Non-DoH) em 3 partições, assim como nós. Essa divisão nos levou à proporção 15:1:18 para cada um desses 3 subconjuntos antes do oversampling com SMOTE, que igualou a classe minoritária (Benign-DoH) à Malicious-DoH. Após o balanceamento, chegamos a 1:1:1, em contraste aos 15:12:12 obtido pelos autores.

2) Pré-processamento no Nosso Experimento: 8 Classes

Neste experimento, lidamos com uma proporção mais complexa de se trabalhar com: 45:8:2:2:2:2:1:1. Mais complexa porque dividir as instâncias majoritárias (Non-DoH) em apenas 3 conjuntos ainda resultaria em um desbalanceamento considerável e precisaríamos gerar muitas instâncias sintéticas para muitas classes diferentes. Nossa estratégia foi, portanto, dividir as instâncias Non-DoH em 5 partições em vez de 3, o que nos levou à proporção 9:8:2:2:2:2:1:1 para cada subconjunto. Após a aplicação do SMOTE utilizando como sampling strategy a quantidade de instâncias da classe dns2tcp (segunda classe majoritária), chegamos à proporção de 1:1:1:1:1:1:1:1.

3) Modelos:

Tanto para a replicação da metodologia original do artigo de referência no novo *dataset*, quanto na nossa proposta de experimento, cada uma das partições de dados do conjunto de treino foi alimentada a um modelo de Random Forest diferente, todos treinados com os mesmos parâmetros: `n_estimators=10`, `max_features=28`, e `random_state=42`. Dessa forma, no contexto em que houve divisão dos dados de treino em 3 subconjuntos (reprodução da metodologia em um *dataset* novo), 3 diferentes Random Forests foram treinados; naquele em que foi realizada a divisão dos dados de treino em 5 subconjuntos (nossa proposta de melhoria), 5 diferentes Random Forests foram treinados. Em seguida, as probabilidades geradas por cada Random Forest alimentam um meta-modelo, que é uma Regressão Logística. Esse passo a passo segue a mesma estratégia do trabalho original, diferindo apenas no segundo experimento, já que o meta modelo recebe as probabilidades de 5 classificadores (já que existem 5 subconjuntos de dados) em vez de 3. Os

parâmetros utilizados para o treinamento do regressor foram `multi_class='multinomial'`, `max_iter=1000`, `random_state=42`.

D. Métricas de avaliação

Para a avaliação dos modelos, utilizamos 4 métricas quantitativas: acurácia geral, e precisão, recall, e f1-score obtidos para cada classe, calculadas utilizando a mesma estratégia do artigo de referência, conforme previamente especificado neste relatório. Além disso, fizemos também uma análise de explicabilidade utilizando a biblioteca SHAP, especialmente através do plot de *feature importances*, para entendermos se os critérios de tomada de decisão dos modelos se mantêm consistentes em contextos de classificação diferentes do que foi trabalhado no artigo de referência.

E. Modelo baseline: Random Forest

Como forma de estabelecer uma linha de base para avaliação do modelo proposto, foi implementado um classificador Random Forest simples, sem a utilização de técnicas de ensemble avançadas, como stacking (implementadas no artigo e no novo conjunto) e sem a utilização de técnicas de *oversampling* como SMOTE. Essa configuração permite isolar o desempenho do modelo em um cenário mais direto, servindo como uma referência para comparações com abordagens mais sofisticadas.

O modelo Random Forest foi configurado utilizando hiperparâmetros padrão, garantido simplicidade e reprodutibilidade dos experimentos.

- **random_state:** 42
- **n_estimators:** 100
- **criterion:** "gini"
- **max_depth:** None
- **min_samples_split:** 2
- **min_samples_leaf:** 1
- **max_features:** sqrt
- **bootstrap:** True

Esse modelo baseline permite avaliar os ganhos obtidos pela abordagem proposta, evidenciando o impacto do uso de técnicas mais sofisticadas na detecção de tráfego malicioso em ambientes DoH.

VII. RESULTADOS E DISCUSSÕES

A reprodução fiel da arquitetura Balanced Stacked Random Forest, treinada e avaliada estritamente sobre o CIRA-CIC-DoHBrw-2020 com o pipeline descrito na Seção VI, obteve os resultados apresentados na Tabela III

A. Solução do artigo de referência

Apesar da queda observada na classe Benign-DoH, o desempenho global permaneceu elevado. Considerando as três classes do problema, o modelo reproduzido manteve precisão, recall e F1-score ponderados próximos de 98%, indicando que a arquitetura continua altamente eficaz para a tarefa de classificação de tráfego DoH. A principal divergência em relação ao artigo original concentrou-se em uma única classe minoritária, e não em uma degradação generalizada do

Tabela III
COMPARAÇÃO DE DESEMPENHO: ARTIGO ORIGINAL VS. REPRODUÇÃO

Métrica	Original (Zebin, 2022)	Reprodução (Este trabalho)
Precisão (geral)	99,91%	98%
Recall (geral)	99,92%	98%
F1-score (geral)	99,91%	98%
Recall: Malicious-DoH	≈ 99,9%	98%
Recall: Non-DoH	≈ 99,8%	99%
Recall: Benign-DoH	≈ 97,3%	45%

sistema, distinção relevante para a interpretação correta dos resultados.

A discrepância mais expressiva concentra-se, de fato, inteiramente na classe minoritária. Enquanto a detecção de tráfego malicioso permaneceu robusta na reprodução (98% de recall, contra os 99,9% relatados pelos autores), o recall da classe Benign-DoH caiu de aproximadamente 97,3% para apenas 45%. Em termos absolutos, dos 1.975 fluxos Benign-DoH no conjunto de teste, apenas 887 foram corretamente identificados; 945 foram classificados como Non-DoH e 143 como Malicious-DoH.

É relevante notar que o padrão qualitativo desse erro é consistente com o que os próprios autores do artigo original relatam: eles também identificam a confusão entre Benign-DoH e Non-DoH como a principal fonte de erro do modelo, atribuindo-a à semelhança entre essas duas classes em características como duração e comprimento de pacote. Ou seja, a reprodução não revelou um comportamento estrutural diferente do modelo (a fronteira de decisão aprendida confunde as mesmas classes que o modelo original confunde), mas sim uma diferença de magnitude muito mais acentuada nesse erro. Isso sugere que detalhes de implementação, configuração experimental ou variações inerentes ao processo de treinamento podem ter influenciado os resultados reportados pelos autores, embora não seja possível, a partir de uma única reprodução, isolar a causa específica dessa diferença, uma investigação mais conclusiva exigiria testar múltiplas sementes aleatórias e configurações de hiperparâmetros, o que está fora do escopo deste trabalho.

Do ponto de vista de um sistema de segurança em produção, a capacidade de detectar o tráfego efetivamente malicioso (a função de defesa primária do sistema) foi preservada com alta fidelidade, enquanto a capacidade de diferenciar tráfego benigno-DoH de tráfego não-DoH (uma distinção de menor criticidade de segurança) foi a que mais se degradou.

Além das métricas de classificação, a reprodução permitiu validar qualitativamente os mecanismos de explicabilidade propostos pelos autores. A análise das importâncias globais geradas pelo dashboard SHAP confirma parcialmente o padrão reportado pelos autores: as oito *features* mais relevantes na réplica pertencem majoritariamente às categorias de comprimento de pacote (Packet Length) e variância de tempo de pacote (Packet Time), mesmas categorias destacadas no artigo original como as mais discriminativas após a duração do fluxo.

Há, no entanto, uma divergência na posição relativa da *feature* Duration: enquanto os autores a reportam como o preditor isoladamente mais importante do modelo, na réplica ela ocupa a 7ª posição, superada por seis *features* de comprimento e tempo de pacote. Essa diferença sugere que o sub-modelo explicado pelo dashboard pode ter aprendido a se apoiar mais fortemente em estatísticas de forma do pacote do que em duração para discriminar entre as três classes, sem que isso comprometa a coerência geral do conjunto de *features* relevantes identificado.

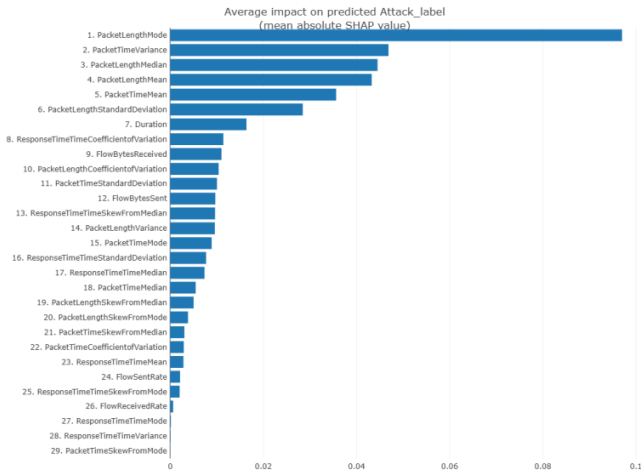


Fig. 2. Gráfico de importância de *features* gerado pelo dashboard da réplica.

B. Proposta da equipe para melhorar a solução de referência

Nos dados usados pelo artigo de referência: Diante do baixo recall observado para Benign-DoH, a equipe testou a variante descrita na Seção V, a aplicação de *class_weight* = 'balanced' (Equação 1) ao meta-classificador de Regressão Logística, mantendo inalterada a camada de sub-modelos Random Forest. Os resultados dessa variante, sobre o mesmo conjunto de teste, são comparados à configuração purista na Tabela IV.

Tabela IV
ANÁLISE DE IMPACTO COM PESOS DE CLASSE NA VARIANTE BENIGN-DOH

Variante	Recall	Precisão	F1-score	Falsos Positivos
Artigo original	45%	83%	0,58	178
<i>class_weight</i> ='balanced'	92%	22%	0,36	6.393

O ajuste elevou o recall de Benign-DoH de 45% para 92%, uma melhora expressiva na capacidade do modelo de identificar tráfego benigno-DoH, mas ao custo de uma queda acentuada da precisão, de 83% para 22%. Na prática, isso significa que o modelo passou a sinalizar milhares de fluxos de Non-DoH e Malicious-DoH como Benign-DoH indevidamente, um volume de falsos positivos que, em um cenário operacional real de um centro de operações de segurança (SOC), geraria fadiga de alertas e tenderia a ser descartado

pelos analistas efetivamente neutralizando o ganho de recall obtido.

Diante desse trade-off severo entre recall e precisão, a equipe optou por manter a arquitetura fiel à descrição original dos autores como o modelo final reportado, tratando o experimento com *class_weight* = 'balanced' como uma investigação diagnóstica e não como uma substituição válida do modelo original. Essa decisão evidencia que o desafio de reprodutibilidade identificado não é resolvido por um ajuste simples de hiperparâmetro: o desbalanceamento extremo do CIRA-CIC-DoHBrw-2020 (45:1:12) impõe um limite estrutural à capacidade de qualquer ajuste de peso de classe equilibrar recall e precisão simultaneamente para a classe minoritária. Esses resultados não sugerem necessariamente uma falha de implementação, mas evidenciam as dificuldades práticas de generalização da classe minoritária sob um regime rigoroso de separação treino/teste, mesmo com a aplicação de técnicas de balanceamento como o SMOTE.

C. Resultados reprodução do artigo de referência

- 1) Nos dados usados pelo artigo de referência
- 2) **No novo conjunto de dados escolhido**

Ao replicarmos a metodologia utilizada no artigo de referência no *dataset* CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD, obtivemos uma acurácia geral de 99%. Entretanto, a acurácia por si só é uma métrica menos relevante para o cenário de detecção de ataques DoH, especialmente em um *dataset* desbalanceado. Por isso, também analisamos as outras métricas em relação a cada classe em vez de utilizarmos um valor agregado, como ilustra a Tabela V.

Tabela V
MÉTRICAS DE AVALIAÇÃO DE DESEMPENHO DO MODELO STACKING ENSEMBLE PARA O PRIMEIRO EXPERIMENTO COM O *dataset* CIRA-CIC-DOHBRW-2020-AND-DOH-TUNNEL-TRAFFIC-HKD (3 CLASSES)

Classe	Precision	Recall	F1
Benign-DoH	0.758	0.925	0.833
Malicious-DoH	0.999	0.999	0.999
Non-DoH	0.998	0.994	0.996

É possível perceber que a predição da classe minoritária (Benign-DoH) foi uma tarefa um pouco mais desafiadora para o modelo em comparação às outras duas classes, inclusive com resultados inferiores aos obtidos originalmente pelos autores. Ainda assim, a performance observada para a classe Malicious-DoH é mais relevante para o problema de detecção discutido neste trabalho, já que em uma aplicação real, seria muito mais valioso que o sistema priorizasse a identificação de fluxos de rede maliciosos em detrimento dos benignos ou Non-DoH. Mesmo assim, pode-se inferir com base nas métricas quantitativas analisadas, que a adição de novas instâncias maliciosas aos dados aumentou o desbalanceamento do *dataset* ao ponto de prejudicar os resultados da

classificação, indicando que a metodologia proposta pode precisar de ajustes e adaptações para lidar com diferentes cenários de desbalanceamento.

Em relação à explicabilidade, podemos perceber a partir do plot de *feature importances* (Figura 3) gerado através da biblioteca SHAP, que as *features* "Duration" e "PacketLengthMode" permanecem como as mais importantes para a tomada de decisão do modelo para o novo *dataset* (em comparação com o CIRA-CIC-DoHBrw-2020), apenas em posições trocadas no ranking de importância. Esse é um indicativo de que, mesmo com a inclusão de novas instâncias maliciosas geradas por novas ferramentas de *tunneling*, a identificação de fluxo de rede malicioso no contexto em análise é pouco dependente de qual ferramenta o gerou.

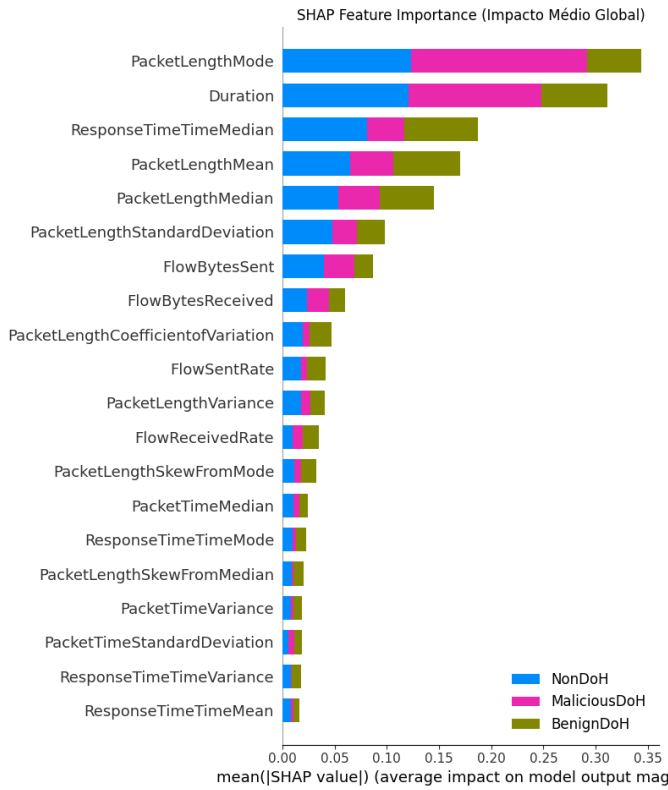


Fig. 3. *Feature Importances* para o experimento de classificação com 3 classes

D. Resultados proposta de melhoria do artigo de referência

Também alcançamos uma acurácia geral de 99% ao avaliar os modelos treinados conforme nossa proposta de melhoria, mas de forma similar à análise anteriormente realizada sobre a reprodução da metodologia original do artigo de referência, a acurácia não é a melhor métrica para avaliar o sistema trabalhado neste projeto. Na Tabela VI, encontramos os valores de precisão, recall e f1 detalhados para cada classe.

Tabela VI
MÉTRICAS DE AVALIAÇÃO DE DESEMPENHO DO MODELO STACKING
ENSEMBLE PARA O SEGUNDO EXPERIMENTO COM O *dataset*
CIRA-CIC-DoHBrw-2020-AND-DoH-TUNNEL-TRAFFIC-HKD (8
CLASSES)

Classe	Precision	Recall	F1
Benign-DoH	0.836	0.917	0.875
Malicious-DoH-dns2tcp	0.997	0.979	0.988
Malicious-DoH-dnscat2	0.871	0.93	0.899
Malicious-DoH-dnstt	0.999	1.0	1.0
Malicious-DoH-iodine	0.921	0.937	0.929
Malicious-DoH-tcp-over-dns	1.0	1.0	1.0
Malicious-DoH-tuns	1.0	0.999	1.0
Non-DoH	0.998	0.996	0.997

Assim como no experimento anterior, a classe Benign-DoH, que é a minoritária, foi a que obteve os piores resultados, embora a precisão e o recall tenham alcançado um desempenho melhor. A nova estratégia de balanceamento pode, de fato, ter contribuído para essa melhoria.

Em relação à identificação das ferramentas de tunneling em específico, é possível observar uma performance um pouco inferior para os ataques criados através da dnscat2 e iodine. À primeira vista, pode parecer que o desbalanceamento possui uma influência considerável nessa queda de desempenho, mas dentre todas as ferramentas de tunneling, as instâncias geradas com iodine são a segunda classe majoritária – o que enfraquece essa hipótese, embora não a invalide.

Uma observação interessante e que pode ajudar a explicar o comportamento é que os resultados obtidos para os ataques gerados com iodine e dnscat2, que os autores do artigo original identificam como tendo comportamentos parecidos na análise de explicabilidade, foram os ataques que segundo a nossa análise, obtiveram os piores resultados. Isso ajuda a indicar que, de fato, o comportamento das duas ferramentas de tunneling pode ser parecido. Já em relação ao *plot* de *feature importances*, conforme observa-se na Figura 4, "Duration", que era um dos atributos mais importantes para a tomada de decisão dos modelos nos experimentos anteriores foi considerado um dos menos importantes. Por outro lado, "PacketLengthMode" permanece no topo de importância. Isso pode indicar que embora a duração do fluxo de rede ajude a identificar se uma instância é um ataque ou não, ela não é decisiva na identificação da ferramenta que gerou o ataque (para afirmar isso, mais experimentos com outros conjuntos de dados são necessários).

E. Resultados do Modelo Baseline

Os resultados obtidos indicam que o modelo apresenta desempenho satisfatório nas métricas gerais de classificação, demonstrando sua capacidade de identificar padrões básicos no tráfego de rede.

No entanto ao comparar com a abordagem proposta, observa-se que o modelo baseline apresenta limitações na identificação de padrões mais complexos.

Em contraste, a abordagem proposta demonstrou melhor capacidade de generalização e maior robustez, evidenciando

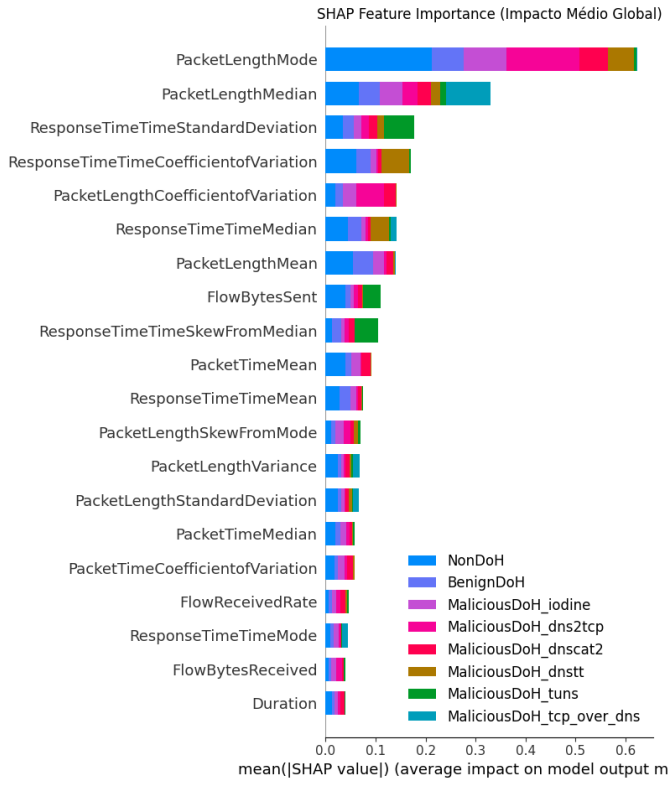


Fig. 4. *Feature Importances* para o experimento de classificação com 8 classes

a importância do uso de técnicas mais avançadas, utilizadas no artigo base[10].

Tabela VII
COMPARAÇÃO DE DESEMPENHO: ARTIGO ORIGINAL VS. RANDOM FOREST

Métrica	Original (Zebin, 2022)	Random Forest (Simples)
Precisão (geral)	99,91%	98%
Recall (geral)	99,92%	98%
F1-score (geral)	99,91%	98%
Recall: Malicious-DoH	≈ 99,9%	100%
Recall: Non-DoH	≈ 99,8%	97%
Recall: Benign-DoH	≈ 97,3%	97%

VIII. CONCLUSÃO, LIMITAÇÕES E TRABALHOS FUTUROS

A. Conclusão

Este trabalho reproduziu e avaliou o sistema de detecção de tráfego DNS over HTTPS malicioso proposto por [2] em dois cenários: no conjunto de dados original e em um *dataset* ampliado com mais instâncias maliciosas. Essa abordagem permitiu tanto verificar a reprodutibilidade dos resultados quanto analisar o comportamento da arquitetura em uma distribuição de dados mais diversa.

No *dataset* original, a arquitetura manteve alto desempenho na tarefa principal, com aproximadamente 98% de precisão,

recall e F1-score ponderados, além de 98% de recall para a classe Malicious-DoH. A principal divergência em relação ao artigo ocorreu na classe Benign-DoH, cujo recall caiu para 45%, indicando maior sensibilidade do modelo a variações no processo de treinamento e às características do conjunto de dados.

No *dataset* ampliado, tanto a reprodução da abordagem original quanto a proposta da equipe apresentaram desempenho global elevado, com cerca de 99% de acurácia. Observou-se também uma melhora significativa na classe Benign-DoH, com F1-scores de 0,833 e 0,875, respectivamente, sugerindo que a maior diversidade dos dados contribuiu para uma melhor generalização do modelo.

A análise via SHAP indicou consistência nas *features* mais relevantes entre os cenários avaliados voltados à classificação simples entre Non-DoH, Malicious-DoH, e Benign-DoH, com destaque para atributos relacionados ao comprimento e ao comportamento temporal dos pacotes, especialmente PacketLengthMode, reforçando a estabilidade dos padrões aprendidos pelo modelo mesmo em configurações mais complexas. Entretanto, a duração do fluxo de rede foi pouco relevante na identificação da ferramenta de tunneling responsável por gerar o ataque.

De forma geral, os resultados confirmam a eficácia da arquitetura proposta para detecção de tráfego DoH malicioso e evidenciam que a qualidade e diversidade dos dados exercem influência mais significativa sobre o desempenho das classes minoritárias do que ajustes isolados de hiperparâmetros.

B. Principais limitações do artigo de referência

Apesar dos resultados positivos, algumas limitações devem ser consideradas. O *dataset* original foi construído a partir de um conjunto específico de navegadores e ferramentas de tunelamento DNS, o que pode limitar a generalização para outros ambientes e aplicações. O forte desbalanceamento entre as classes permanece um desafio central: mesmo com SMOTE e subconjuntos balanceados, a classe Benign-DoH mostrou-se consistentemente mais difícil de generalizar do que as demais, e ajustes simples de pesos de classe não resolvem o problema sem introduzir novos erros críticos. O sistema também depende da extração prévia de características estatísticas dos fluxos, o que condiciona seu desempenho à qualidade e disponibilidade dessa etapa em ambientes reais. Por fim, parte do processo de stacking descrito pelos autores não é documentada em detalhes suficientes para garantir uma reprodução completamente determinística, e o dashboard XAI foi construído sobre um único sub-modelo da camada base, refletindo a lógica de apenas um dos classificadores e não a do meta-classificador responsável pela decisão final.

C. Principais limitações da proposta de melhoria

Embora a proposta de melhoria tenha apresentado resultados superiores em relação ao cenário original, algumas limitações devem ser consideradas. Como as modificações na estratégia de balanceamento e no conjunto de dados foram realizadas simultaneamente, não foi possível determinar precisamente a

contribuição individual de cada alteração para os ganhos observados. Além disso, a estratégia baseada em cinco subconjuntos não foi comparada diretamente com a abordagem original no mesmo conjunto de dados, o que limita conclusões mais definitivas sobre sua efetividade. Também permanecem oportunidades de investigação relacionadas à distinção entre ferramentas com padrões estatísticos semelhantes (como iodine e dnscat2), à ampliação das análises de explicabilidade para o nível de instância individual e à avaliação da estabilidade dos resultados por meio de múltiplas execuções independentes

D. Trabalhos futuros

Com base nos resultados obtidos e nas limitações identificadas ao longo da reprodução, diversas oportunidades de investigação podem ser exploradas em trabalhos futuros. Uma direção promissora consiste na avaliação do modelo em cenários mais próximos de ambientes reais, incluindo aplicações móveis, sistemas operacionais com suporte nativo ao protocolo DoH e diferentes perfis de usuários, permitindo analisar a capacidade de generalização da solução em contextos mais diversos. Além disso, podem ser investigadas arquiteturas de aprendizado profundo, capazes de capturar padrões temporais presentes nos fluxos de rede que não são adequadamente representados pelas características estatísticas agregadas utilizadas neste trabalho.

Outra possibilidade consiste em expandir a análise para outros tipos de ameaças associadas ao protocolo DoH, incluindo tráfego relacionado a Domain Generation Algorithms (DGAs), botnets e outras formas de comunicação maliciosa baseadas em DNS. Também se mostra relevante estudar a adaptação da solução para ambientes operacionais em tempo real, integrando os processos de extração de características, classificação e geração de alertas a sistemas contínuos de monitoramento de rede. Adicionalmente, a realização de experimentos utilizando múltiplas sementes aleatórias e diferentes configurações de treinamento pode fornecer uma avaliação estatística mais robusta da variabilidade dos resultados e da estabilidade do modelo. Por fim, futuras pesquisas podem investigar mecanismos de explicabilidade aplicados diretamente à arquitetura completa de stacking, permitindo que as interpretações produzidas estejam alinhadas à decisão efetivamente gerada pelo sistema final e ampliando a transparência do processo de classificação.

REFERÊNCIAS

- [1] International Data Corporation (IDC), “Idc 2021 global dns threat report,” <https://efficientip.com/resources/idc-dns-threat-report-2021/>, 2021, iDC InfoBrief sponsored by EfficientIP. Accessed: 2026-06-22.
- [2] T. Zebin, S. Rezvy, and Y. Luo, “An explainable ai-based intrusion detection system for dns over https (doh) attacks,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2339–2349, 2022.
- [3] L. A. Trejo *et al.*, “Dns-advp: A machine learning anomaly detection and visual platform to protect top-level domain name servers against ddos attacks,” *IEEE Access*, 2019.
- [4] D. Vekshin, K. Hynek, and T. Cejka, “Doh insight: Detecting dns over https by machine learning,” in *ARES*, 2020.
- [5] M. T. Jafar *et al.*, “Analysis and investigation of malicious dns queries using cira-cic-dohbrw-2020 dataset,” 2021, 2021.
- [6] L. Nussbaum, P. Neyron, and O. Richard, “On robust covert channels inside dns,” in *IFIP International Information Security Conference*, 2009, pp. 51–62.
- [7] L. Salat, M. Davis, and N. Khan, “Dns tunnelling, exfiltration and detection over cloud environments,” vol. 23, no. 5, 2023, p. 2760.
- [8] D. Fifield, “Dnsth: Dns tunnel over doh and dot,” <https://www.bamssoftware.com/software/dnsth/>, 2020, acessado em: 22 jun. 2026.
- [9] E. Valente, A. Osti, L. P. Júnior, and J. Estrella, “Doh deception: Evading ml-based tunnel detection models with real-world adversarial examples,” in *Anais do XXIV Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais*. Porto Alegre, RS, Brasil: SBC, 2024, pp. 287–302. [Online]. Available: <https://sol.sbc.org.br/index.php/sbseg/article/view/30032>
- [10] M. MontazeriShatoori, L. Davidson, G. Kaur, and A. Habibi Lashkari, “Detection of doh tunnels using time-series classification of encrypted traffic,” in *2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech)*, 2020, pp. 63–70.
- [11] R. Mitsuhashi, Y. Jin, K. Iida, T. Shinagawa, and Y. Takai, “Malicious dns tunnel tool recognition using persistent doh traffic analysis,” *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 2086–2095, 2023.
- [12] —, “Cira-cic-dohbrw-2020 and doh-tunnel-traffic-hkd combined dataset,” <https://github.com/doh-traffic-dataset/CIRA-CIC-DoHBrw-2020-and-DoH-Tunnel-Traffic-HKD>, 2023, gitHub repository. Accessed: 2025-06-21.